
Software Requirements Specification

for

<EasyGo>

Version 1.0 approved

Prepared by <Andy Thiang, Justin Tan, Dinh Quang Anh, Caasi John
Rian Domingo, Benjamin Phang, Chew Hong Ming>

<SC2006 - EasyGo>

<2/11/2025>

Table of Contents

1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	2
1.5 References	2
2. Overall Description	3
2.1 Product Perspective	3
2.2 Product Functions	4
2.2.1 Use Case Diagram	4
2.2.2 Major Product Functions	5
2.2.3 Class Diagram	6
2.3 User Classes and Characteristics	7
2.4 Operating Environment	7
2.5 Design and Implementation Constraints	8
2.5.1 Frontend Constraints	8
2.5.1.1 Framework (Frontend)	8
2.5.1.2 Code Formatting	8
2.5.2 Backend Constraints	8
2.5.2.1 Framework (Backend)	8
2.5.2.2 Database	8
2.5.2.3 Code Formatting	8
2.5.3 Language Constraints	8
2.6 User Documentation	9
2.6.1 README Files	9
2.6.1.1 Main Repository	9
2.6.1.2 Project Repository	9
2.6.2 Code Comments	9
2.6.3 Design Overview	9
2.7 Assumptions and Dependencies	10
2.7.1 Assumptions	10
2.7.2 Dependencies	10
2.7.2.1 Libraries	10
2.7.2.2 External Services	11
3. External Interface Requirements	12
3.1 User Interfaces	12
3.1.1 Style Guides	12
3.1.1.1 Theme Colours	12
3.1.2 UI Mockups	12

3.1.2.1 Authentication(sign in and sign up page)	13
3.1.2.2 Landing Page	15
3.1.2.3 Route Page (with convenience score)	16
3.1.2.4 Saved Route page	18
3.2 Hardware Interfaces	19
3.2.1 Device Type	19
3.2.2 Data and Control Interactions	19
3.2.3 Communication Protocols	19
3.3 Software Interfaces	20
3.3.1 Operating System	20
3.3.2 Web Browsers	20
3.3.3 Databases	20
3.3.4 Backend & Frontend Tools	20
3.3.5 List of APIs used	20
3.3.6 Data Sharing	20
3.4 Communications Interfaces	21
4. System Features	22
4.1 Search Location Auto-Suggest	22
4.1.1 Description and Priority	22
4.1.2 Stimulus/Response Sequence	22
4.1.3 Functional Requirements	22
4.2 Validate Location	22
4.2.1 Description and Priority	22
4.2.2 Stimulus/Response Sequences	23
4.2.3 Functional Requirements	23
4.3 Select Mode of Transport	24
4.3.1 Description and Priority	24
4.3.2 Stimulus/Response Sequences	24
4.3.3 Functional Requirements	24
4.4 Select Convenience Filter	25
4.4.1 Description and Priority	25
4.4.2 Stimulus/Response Sequences	25
4.4.3 Functional Requirements	25
4.5 Display Convenience Level Score	26
4.5.1 Description and Priority	26
4.5.2 Stimulus/Response Sequences	26
4.5.3 Functional Requirements	26
4.6 Display Interactive Map	27
4.6.1 Description and Priority	27
4.6.2 Stimulus/Response Sequences	27
4.6.3 Functional Requirements	27

4.7 Retrieve Routes	28
4.7.1 Description and Priority	28
4.7.2 Stimulus/Response Sequences	28
4.7.3 Functional Requirements	28
4.8 Save Routes (Itinerary)	29
4.8.1 Description and Priority	29
4.8.2 Stimulus/Response Sequences	29
4.8.3 Functional Requirements	29
4.9 Interactive Map Start/End Point Selector	30
4.9.1 Description and Priority	30
4.9.2 Stimulus/Response Sequences	30
4.9.3 Functional Requirements	30
4.10 Display Driving Route	31
4.10.1 Description and Priority	31
4.10.2 Stimulus/Response Sequences	31
4.10.3 Functional Requirements	31
4.11 Display Nearby Carparks Info	32
4.11.1 Description and Priority	32
4.11.2 Stimulus/Response Sequences	32
4.11.3 Functional Requirements	32
4.12 Display Traffic Incident Info	33
4.12.1 Description and Priority	33
4.12.2 Stimulus/Response Sequences	33
4.12.3 Functional Requirements	33
4.13 Display Public Transit Route	34
4.13.1 Description and Priority	34
4.13.2 Stimulus/Response Sequences	34
4.13.3 Functional Requirements	34
4.14 Display Public Transit Info	35
4.14.1 Description and Priority	35
4.14.2 Stimulus/Response Sequences	35
4.14.3 Functional Requirements	35
4.15 Display Walking Route	36
4.15.1 Description and Priority	36
4.15.2 Stimulus/Response Sequences	36
4.15.3 Functional Requirements	36
4.16 User Login	37
4.16.1 Description and Priority	37
4.16.2 Stimulus/Response Sequences	37
4.16.3 Functional Requirements	37
4.17 Register User	38

4.17.1 Description and Priority	38
4.17.2 Stimulus/Response Sequences	38
4.17.3 Functional Requirements	39
4.18 Change Password	40
4.18.1 Description and Priority	40
4.18.2 Stimulus/Response Sequences	40
4.18.3 Functional Requirements	40
5. Other Nonfunctional Requirements	41
5.1 Performance Requirements	41
5.2 Safety Requirements	41
5.3 Security Requirements	41
5.4 Software Quality Attributes	41
5.5 Business Rules	42
5.5.1 Data Privacy	42
5.5.2 Operational Boundary	42
5.5.3 Transactional Integrity	42
5.5.4 Function Restrictions	43
6. Other Requirements	44
6.1 Internationalisation Requirements	44
6.2 Legal Requirements	44
6.3 Reuse Objectives	44
6.4 Accessibility Requirements	44

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This document outlines the software requirements for the "EasyGo, Version 1.0". This SRS outlines the functionalities, features, and constraints of the platform, aiming to provide a clear and comprehensive framework for development. The scope of this SRS encompasses the entire web application, detailing the subsystems for user management, route generation, and convenience score. This SRS is integral for guiding the development, testing, and maintenance of the platform, ensuring alignment with the project's objectives.

1.2 Document Conventions

This SRS follows standard documentation conventions to ensure clarity and consistency. Key conventions include:

- Font Styles: 'Arial' for body text, 'Times New Roman' for headers.
- Highlighting: Bold for key terms, italics for emphasis.
- Requirement Prioritization: Higher-level requirements have inherited priority levels, cascading down to detailed requirements. Each requirement statement is explicitly labelled with a priority level (High, Medium, Low).
- Requirement Identification: Each requirement is uniquely identified for easy reference.

1.3 Intended Audience and Reading Suggestions

This document should be viewed differently based on the reader's role and be designed to accommodate stakeholders:

- **Project Managers and Stakeholders:** Introductory sections as well as the overall description provides an overview of the system for them to plan and work around.
- **Developers:** After the introduction sections, follow up on the System Features for more detailed descriptions of the platform's functionality and External Interface Requirements for integration specifications.
- **Testers:** Pay more attention to System Features for test cases and refer to Non Functional Requirements for performance and security testing guidelines.

- **User Experience Designers:** The User Interface sections within the External Interface Requirements will be of particular interest, providing details on the interaction between the system and its users.
- **Technical Writers:** Refer to User Documentation, Assumptions and Dependencies to ensure accurate information to provide for documentation, user guides or for a knowledge base.
- **Quality Assurance:** Refer to System Features for functionality, Other NonFunctional Requirements for performance benchmarks and Other Requirements for additional specifications.

1.4 Product Scope

EasyGo is a digital solution designed to generate optimized travel routes between a start and end destination, with a unique focus on user-defined convenience. Its primary goal is to provide users with routes that are not just the shortest or fastest but those that align with their personal preferences and priorities for convenience, thereby enhancing user satisfaction. The application will enable users to input their start and end points and customize convenience factors such as Time Taken, Amount of Walking, Number of Transfers, Crowd Level, Bus Wait Time, Fare Cost and Carpark Availability. This aligns with broader corporate objectives of innovation, personalized digital services, and enhancing user experience in navigation technology.

1.5 References

UI Inspiration: <https://www.google.com/maps>

2. Overall Description

2.1 Product Perspective

This SRS specifies the requirements for EasyGo, a new, self-contained web application designed as a comprehensive transport planner for Singaporean commuters. The system consists of three main subsystems: User Management, Route Generation, and Convenience Score Calculation. EasyGo does not replace an existing system but provides a novel approach to route planning by compiling various transport modes and calculating a unique Convenience Level Score for each route. It serves as a single platform detailing functionalities to guide the development of the product. A system architecture diagram (Figure 2.1.1) is provided to illustrate the connections between the layers and external interfaces.

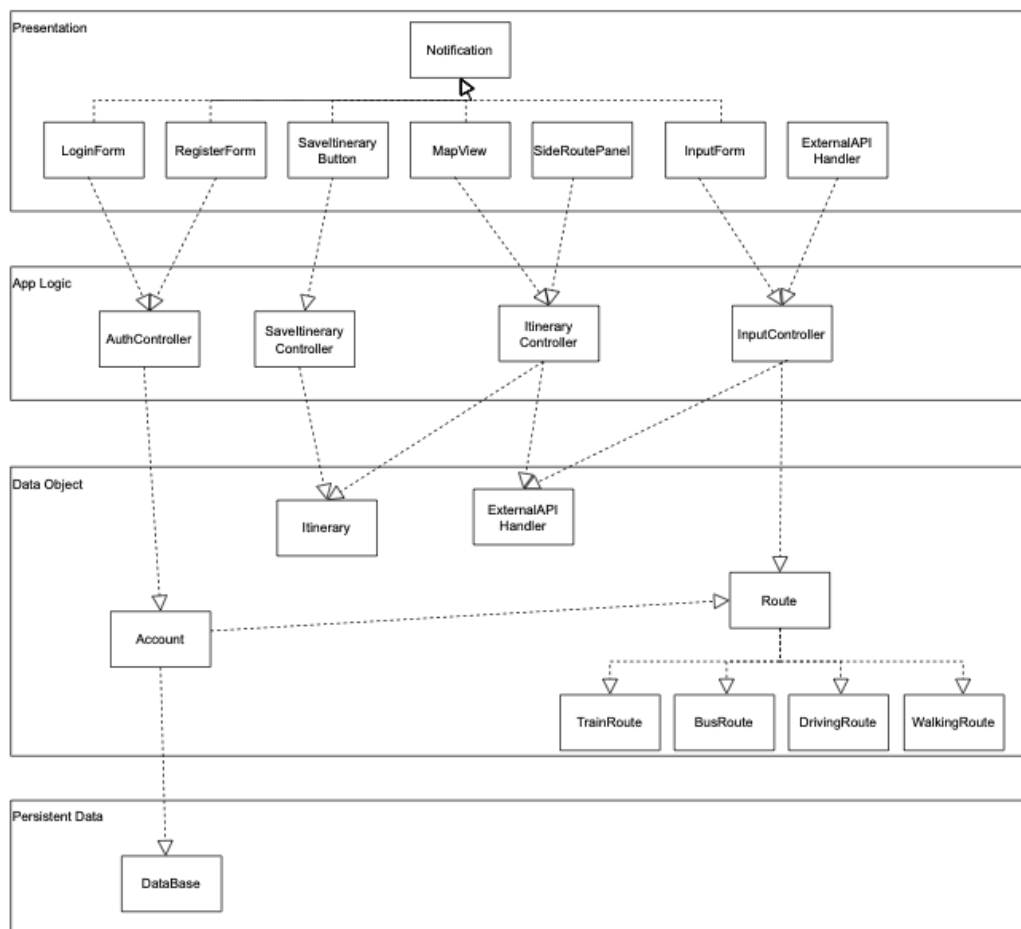


Figure 2.1.1: System Architecture

2.2 Product Functions

2.2.1 Use Case Diagram

The following use case diagram (Figure 2.2.1) depicts the key product functionalities that EasyGo includes. There are 4 primary users of EasyGo - User, OneMap API, Government Data API, and LTA DataMall API. Each type of user interacts with EasyGo to use their role-specific functions.

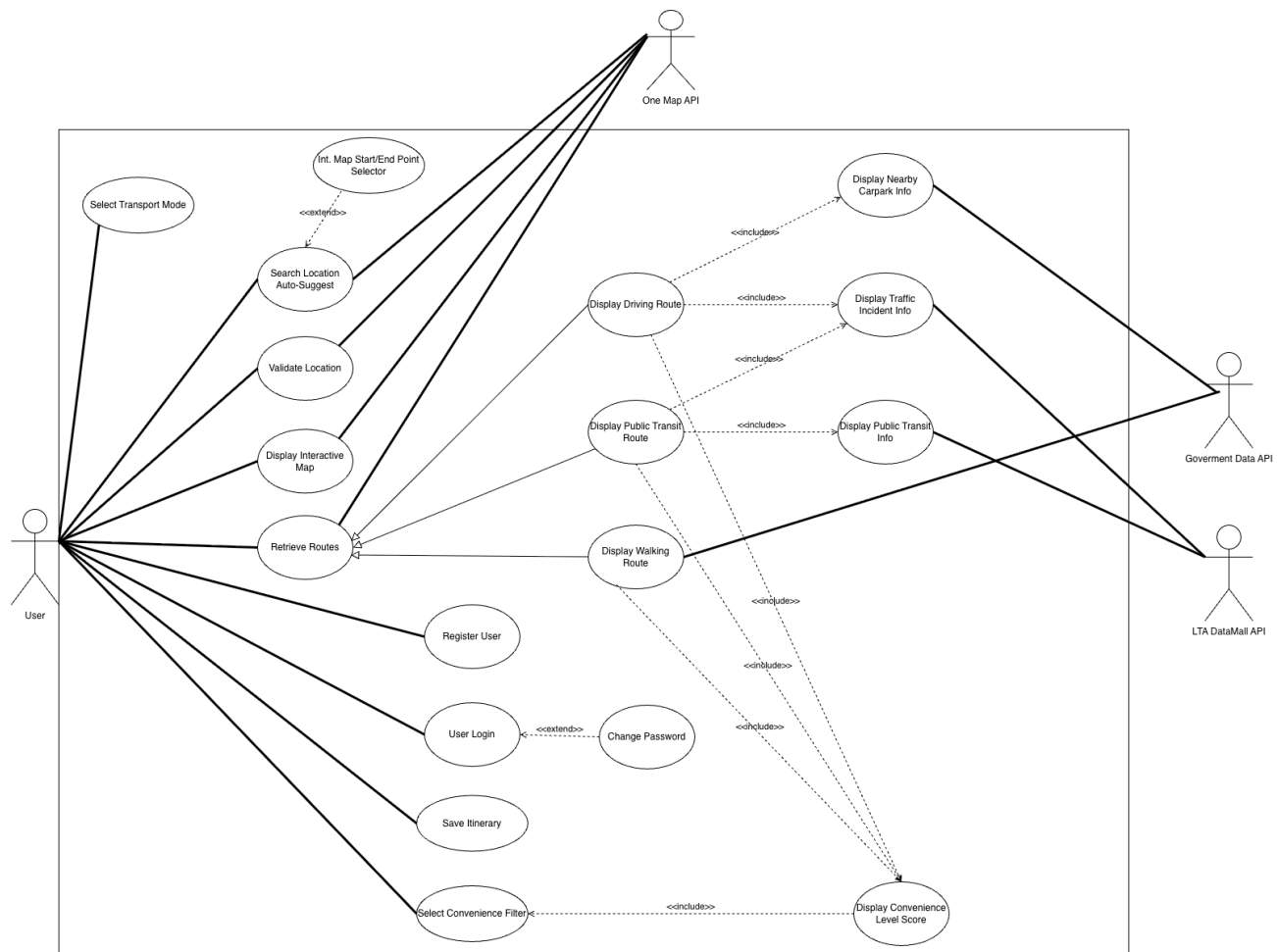


Figure 2.2.1: Use Case Diagram

2.2.2 Major Product Functions

EasyGo will allow users to perform the following functions:

1. Authentication and User Management

- Login
- Register
- Change Password
- Save Routes (Itinerary)

2. Core Route Planning

- Search Location
- Validate Location
- Retrieve Routes
- Display Interactive Map
- Interactive Map Selector

3. Transport Mode Features

- Driving Mode
 - Display Driving Route
 - Display Nearby Carparks
 - Display Traffic Incidents
- Public Transport Mode
 - Display Public Transport Route
 - Bus Arrival Time
 - Crowd Level
- Walking Mode
 - Display Walking Route
 - Walking Distance
 - Walking Time
 - Weather Forecast

4. Utility and Scoring

- Convenience Score Calculation
- Display Best Result
- Filter Selection

5. Other Functions

- Settings
- Edit Profile

2.2.3 Class Diagram

The following Class Diagram (Figure 2.2.3) shows how the different classes interact with one another. It also shows the main functions that EasyGo requires to operate.

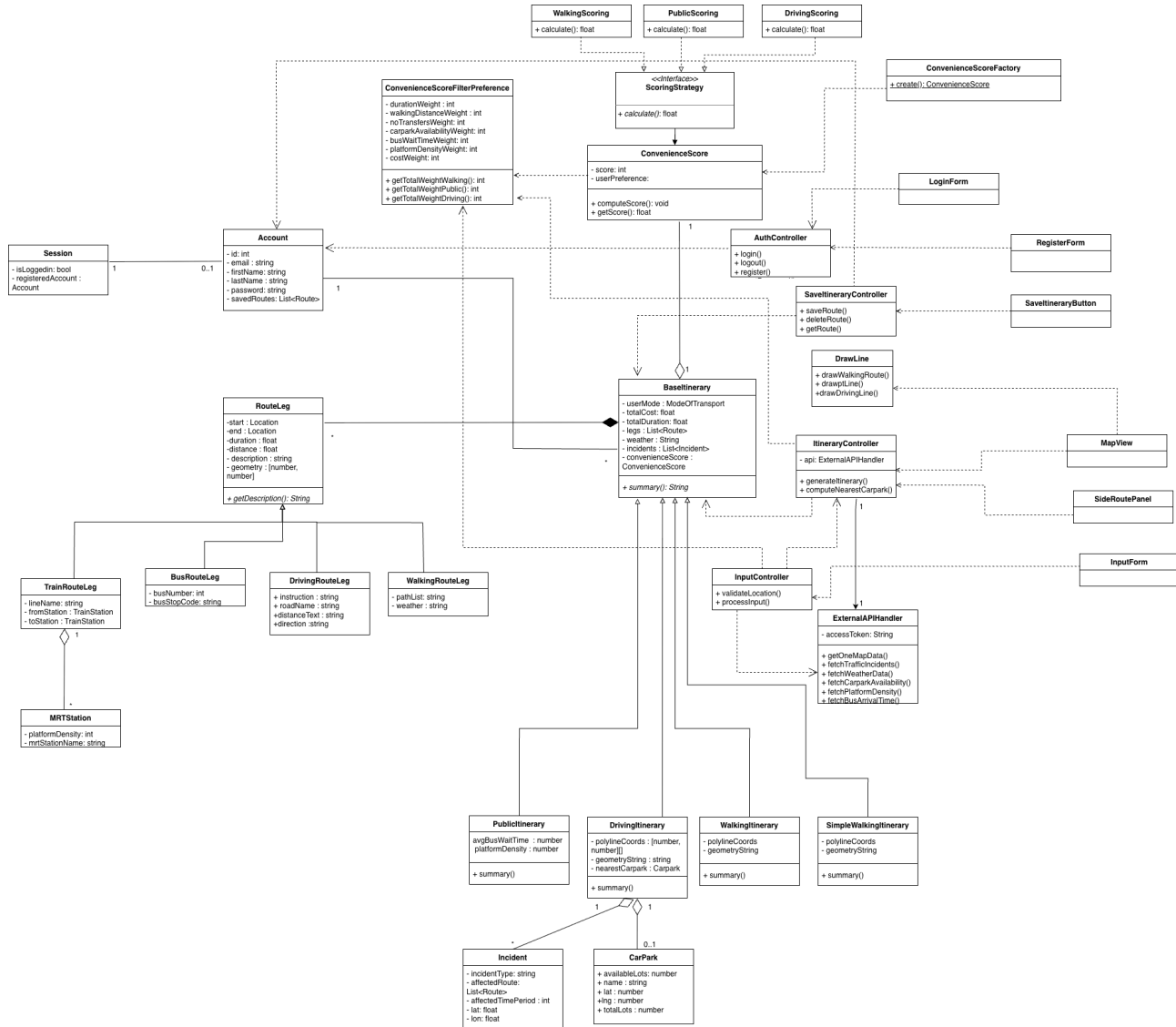


Figure 2.2.3: Class Diagram

2.3 User Classes and Characteristics

The users of the EasyGo application are primarily regular commuters looking for travel routes within Singapore. The user base has one main class based on their privilege and interaction with the system. This user is categorised as:

- **Authenticated User:** A user who has successfully completed the registration and login process. They possess additional privileges beyond the General User.

This user type is vital for EasyGo, as it forms a continuous cycle of usage and personalisation that drives the product's value proposition. E.g. The user accesses the core route planning features, searches for locations, and utilises the real-time data and convenience score output. They are able to register and sign up to transition into an Authenticated User, where they can utilise the additional function of saving their routes for long-term usage.

2.4 Operating Environment

The software will be a web-based operation, optimised for desktop usage, operating on the latest versions of Chrome, Firefox, Safari, and Edge. It will be hosted on cloud services for scalability and reliability. It must also integrate with the external APIs (OneMap, data.gov.sg, LTA DataMall) as well as database storage (Supabase)

2.5 Design and Implementation Constraints

The development of EasyGo will be subjected to several constraints that will guide the design and operation of the software:

2.5.1 Frontend Constraints

2.5.1.1 Framework (Frontend)

The user interface will be developed using React ([Next.js](#)) and TypeScript, with Tailwind CSS for styling. TypeScript is chosen for its strong typing which improves maintainability and reduces runtime errors. [Next.js](#) is used to provide server-side rendering/routing and to simplify integration with API routes.

2.5.1.2 Code Formatting

Code formatting is done using ESLint and Prettier with TypeScript rules. This ensures consistent formatting before commits.

2.5.2 Backend Constraints

2.5.2.1 Framework (Backend)

The framework will be developed using [Node.js](#)/TypeScript. Services will be structured as small service classes (OneMapService, ScoringService, FormatterService) that the Facade/Controller orchestrates.

2.5.2.2 Database

The project uses Supabase for persistent storage of user data, saved routes, and user preferences. Supabase provides Postgres, authentication, and real-time features which fit our accounts and saved-route functionality.

2.5.2.3 Code Formatting

Code formatting is done using ESLint and Prettier with TypeScript rules. This ensures consistent formatting before commits.

2.5.3 Language Constraints

All software documentation, interfaces, and user interactions will be provided in English. The choice of English is due to the prevalence as a primary or secondary language among the anticipated user demographics and to maintain consistency across the platform. This constraint ensures a unified user experience and simplifies the maintenance of all textual elements within the application.

2.6 User Documentation

EasyGo will include a comprehensive set of user documentation to ensure proper understanding and ease of use for both users and developers. The documentation will consist of the following components:

2.6.1 README Files

2.6.1.1 Main Repository

A README file for the [main repository](#) will provide an overview of the project, including the architecture, diagram and explanation. There will also be links to the Frontend, Backend, Docs, and Architecture Files.

2.6.1.2 Project Repository

The repository will include a README File explaining how to run and build the [Next.js](#)/React TypeScript app, the required Node version, and the necessary environment variables. It will also include a short directory map and recommend using the MapDisplay imperative API and the centralised 'useltineraryData' hook for route/state operations. The repository will also include a README file which documents the [Next.js](#) API routes and server-side notes. It will explain the environment variables and point to the controller and scoring code.

2.6.2 Code Comments

- Source files will include clear inline comments and JSDoc/TSDoc headers for public functions, classes, and modules.
- Complex logic (scoring normalisation, platform density calculation, traffic incident matching) will have short inline explanations and assumptions.
- TODO and FIXME tags will be used to mark technical debt and links related to issues where appropriate.

2.6.3 Design Overview

- A short architecture guide will explain the Facade pattern in this project – how the frontend talks to a single facade endpoint, and how the backend strategies are swapped.
- It will also share the strategies and SOLID mapping to provide the next steps for improvements and future expansion.

2.7 Assumptions and Dependencies

2.7.1 Assumptions

It is assumed that the platform will have access to the internet at all times.

2.7.2 Dependencies

2.7.2.1 Libraries

The user interface of EasyGo depends on numerous libraries as follows:

Library	Purpose
Next.js	To run and build the React app with page routing and server-side rendering
React.js	To build the user interface and render it in the browser
Material UI	To use ready-made Material Design UI components (e.g. buttons)
tailwindcss	To style the app using utility classes
tailwind-merge	To safely combine Tailwind utility classes without conflicts
next-themes	To add light/dark theme switching easily in a Next.js app
supabase/supabase-js	To talk to Supabase from the frontend
leaflet	To display interactive maps in the browser
lucide-react	To show a set of vector icons as React components
use-debounce	To debounce values of callbacks inside React components
TypeScript	To provide a strong typing for easier maintainability
ESLint	To catch code style and error problems automatically while coding
Lodash	To perform common data manipulations and utility tasks on the server

2.7.2.2 External Services

EasyGo depends on external services:

- Third-party APIs
 - OneMap API
 - LTA DataMall
 - NEA Weather API
- Cloud hosting services
 - GitHub Pages

In summary, EasyGo relies on the availability and reliability of external services.

3. External Interface Requirements

3.1 User Interfaces

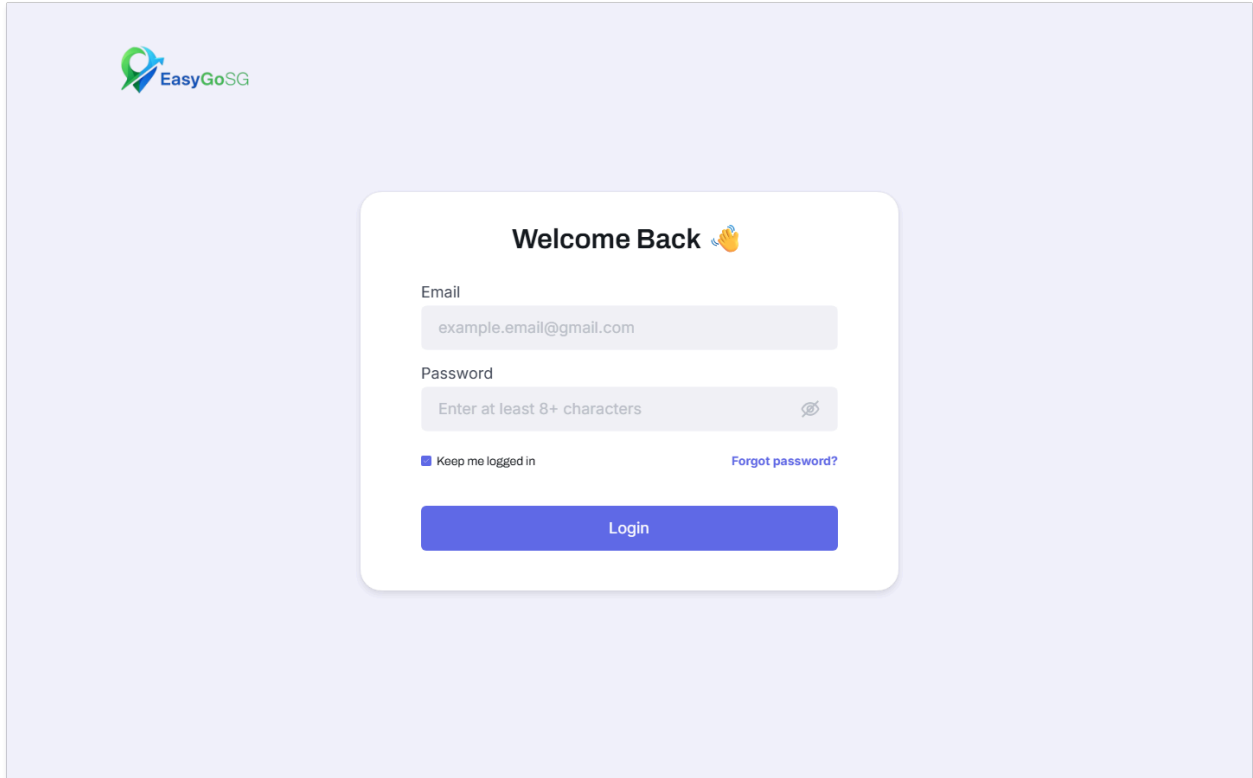
3.1.1 Style Guides

3.1.1.1 Theme Colours

Neutral 0xFFFFFFFF	Karaka 0x32321A	Black 0x000000	Dark Green 0x2B7A4D
Violet 0x2B7A4D	Blue 0x0075B8	Red 0xD9534F	Light Green 0x2B7A4D
Yellow 0xF8C94C	Light Blue 0x2993D9	Grey 0x2993D9	Brown 0x6E4E0B

3.1.2 UI Mockups

3.1.2.1 Authentication(sign in and sign up page)



The image shows a UI mockup for the EasyGoSG authentication page. The page has a light purple background. In the top left corner, there is a logo for EasyGoSG, which consists of a green and blue circular icon followed by the text "EasyGoSG". In the center of the page, there is a white rounded rectangle containing the login form. The form is titled "Welcome Back" with a small orange robot icon. Below the title, there are two input fields: "Email" with the placeholder text "example.email@gmail.com" and "Password" with the placeholder text "Enter at least 8+ characters" and an eye icon. Below the password field, there is a checkbox labeled "Keep me logged in" and a link labeled "Forgot password?". At the bottom of the form, there is a blue button labeled "Login".



Begin your transport journey

First name

Last name

Email

Password

☒ By signing up, I agree with the [Terms of Use](#) & [Privacy Policy](#)

[Register](#)

OR

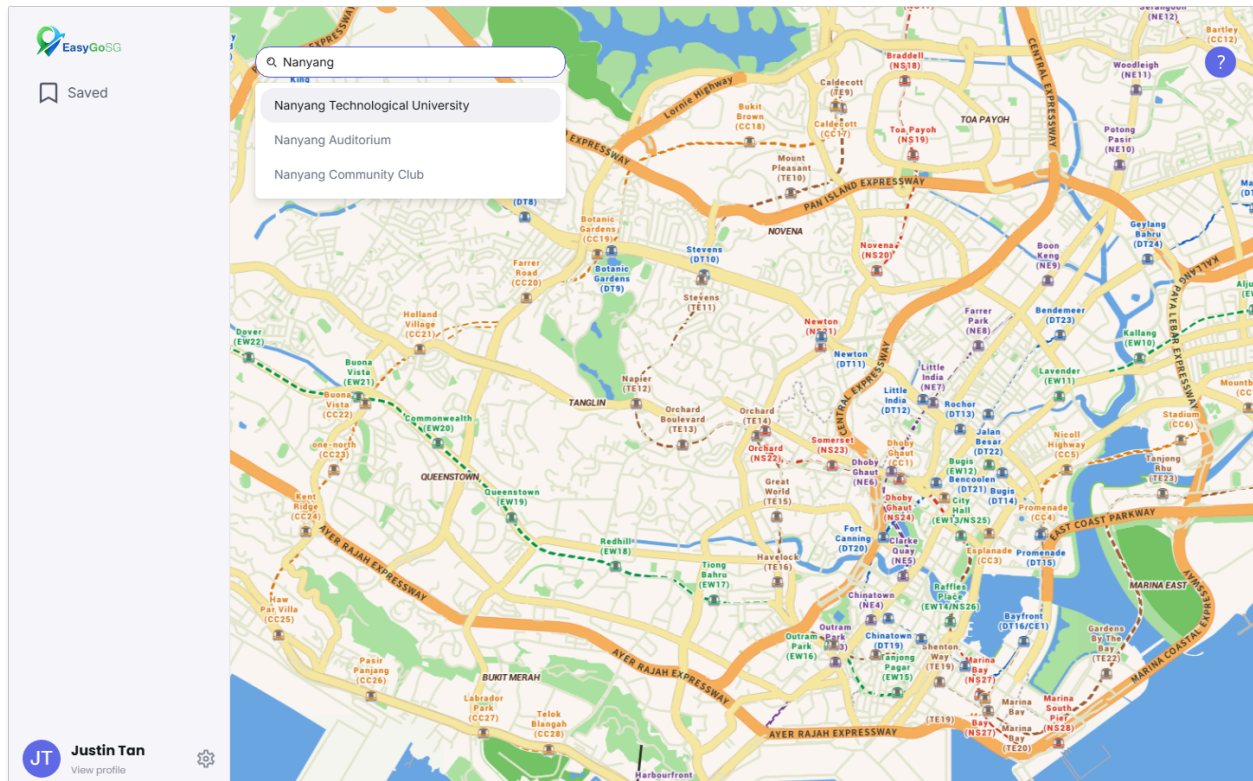
Returning user? [Log in here](#)

Come join us

-  Unlock smarter commutes with Singapore's best transit resources
-  Plan and save your journey with expert routes and transit insights

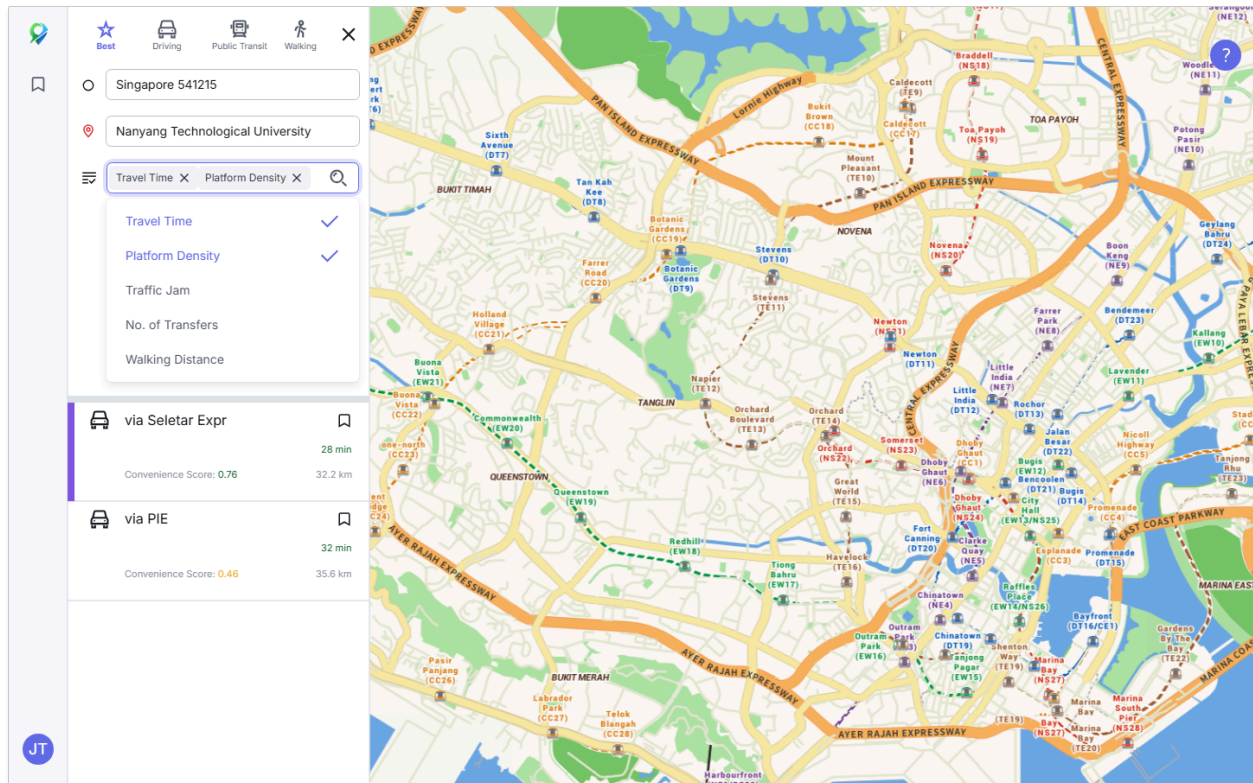
Our web application begins with the signup page. Users will be able to register for an account if they are first-time users. The credentials required are the user's registered email and password. Once the 'Login Page' has been clicked and authenticated, users will be directed to the Landing Page.

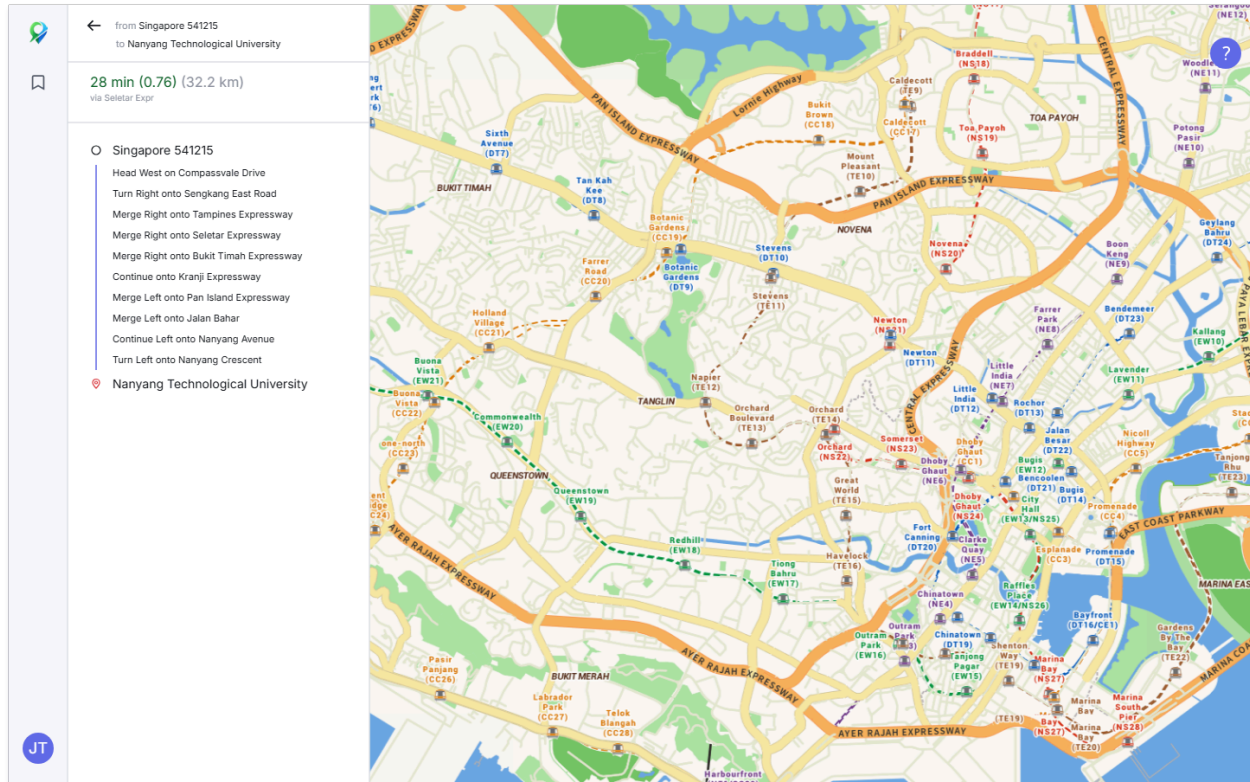
3.1.2.2 Landing Page



At the Landing Page, there will be a search bar for users to be able to select the start and end location of their choice. Once the start and end locations have been imputed and authenticated, users will be directed to the “Route Page”.

3.1.2.3 Route Page (with convenience score)





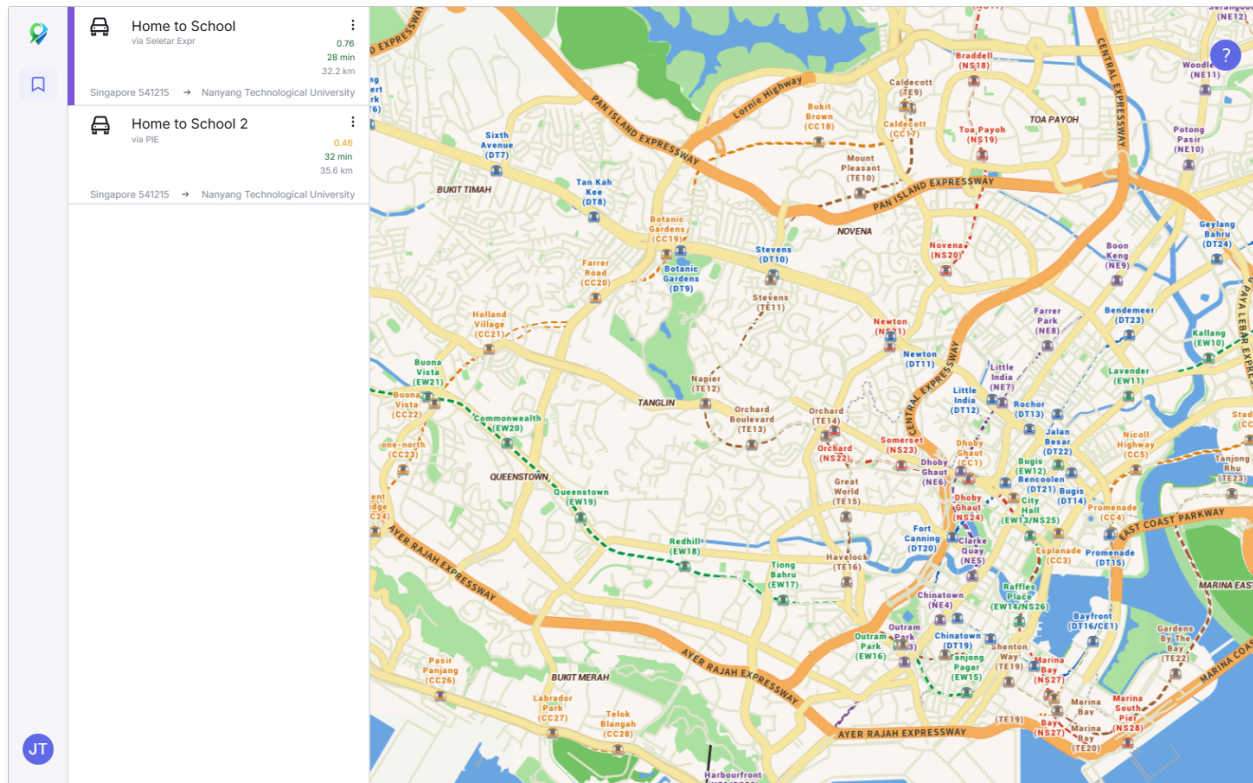
At the route page, the top 3 routes determined by a convenience score system will be displayed in the best tab. More information about the convenience score system can be found in the appendix.

There are also 3 other tabs, consisting of driving, public transport and walking modes. Users can click the respective tabs to view the list of itineraries.

Once the user has selected an itinerary, they can view the directions needed to go to their destination.

Users can also save an itinerary to their account by clicking the save button located next to an itinerary. Saved routes can be viewed in the “Saved Route page”.

3.1.2.4 Saved Route page



At the Saved Route page, users can view past itineraries saved and access them anytime. The travel time and convenience score will be updated live to ensure reliable data.

3.2 Hardware Interfaces

EasyGo is designed to operate primarily as a web-based application, minimizing the need for direct hardware interfaces.

3.2.1 Device Type

The software is designed to be compatible with various devices, including desktop computers, laptops, tablets, and smartphones.

3.2.2 Data and Control Interactions

The user's device interacts with the software through standard input/output hardware such as keyboards, touchscreens, and mice for input; displays for output; and network interface cards for internet connectivity.

3.2.3 Communication Protocols

The platform uses HTTPS for secure web traffic. The APIs, websocket connection, and services will communicate over standard web ports (80 for HTTP and 443 for HTTPS).

3.3 Software Interfaces

3.3.1 Operating System

Any OS capable of running a modern web browser (Windows, macOS, Linux, iOS, Android).

3.3.2 Web Browsers

The latest versions of Chrome, Firefox, Safari, Edge, and others are capable of supporting HTML5, CSS3, and JavaScript.

3.3.3 Databases

SQL-based database systems, accessed through backend services.

3.3.4 Backend & Frontend Tools

1. [Next.js](#)
2. Supabase
3. Shadcn
4. Tailwind CSS
5. Leaflet
6. TypeScript

3.3.5 List of APIs used

1. LTA DataMall API
2. [Data.gov.sg](#) API
3. OneMap API

3.3.6 Data Sharing

JSON format is used for all data exchange between frontend and backend. Data persistence is maintained through the SQL database with specific defined data models.

3.4 Communications Interfaces

For EasyGo, various communication interfaces are integral to ensuring a seamless and responsive user experience:

- **HTTP/HTTPS:** Web applications should be hosted in secure platforms with data transferred over secure HTTPS protocols to secure data channels and compatibility.
- **Network Protocols:** Standard network protocols such as TCP/IP will be used for network communication, ensuring adherence to established internet communication standards.
- **Message Formatting:** To ensure consistency and ease of parsing, WebSocket and communication and API will use standardized message format.
- **Security and Encryption:** TLS (Transport Layer Security) is enforced for all traffic to safeguard data integrity and privacy.
- **API-based Backend Communication:** An internal API layer is used for the asynchronous communication of the backend and front end through JSON, to dynamically update complex UI without requiring a full page reload. This ensures real-time updates and allows smooth navigation for our users.

These communication interfaces are designed to provide secure, efficient and reliable features to enhance the user experience.

4. System Features

Please refer to the 'Use Case Diagram and Descriptions, and Sequence Diagrams.pdf' for the full use case diagram, detailed use case descriptions, and sequence diagrams for each use case.

4.1 Search Location Auto-Suggest

4.1.1 Description and Priority

Enables users to search for Singapore locations using a search bar with auto-suggest functionality. The system will query the OneMap Search API based on the user's input and display a dropdown list of possible matching locations to facilitate the selection of start and end points.

Priority: High

4.1.2 Stimulus/Response Sequence

1. The user enters a partial or full location name in the search bar (e.g. "Orchard" or "Changi Airport").
2. The system sends the user's input to the OneMap Search API in real-time as the user types.
3. The OneMap Search API returns a list of matching Singapore locations based on the input.
4. The system displays the list of locations in an auto-suggest dropdown below the search bar.

4.1.3 Functional Requirements

REQ-1: The system has access to the OneMap Search API.

REQ-2: The user is able to access the search bar interface.

REQ-3: AF-S3: No Matching Locations

1. If the OneMap Search API returns no results for the user's input, the system will display an empty auto-suggest dropdown list.

REQ-4: API Error:

1. If the OneMap Search API is unreachable, no suggestion will be displayed.

4.2 Validate Location

4.2.1 Description and Priority

Validates that the user's submitted start and end locations are valid Singapore locations by querying the OneMap Search API to retrieve their coordinates. The locations are provided as text input in a search bar, and validation ensures they correspond to recognisable Singapore addresses.

Priority: High

4.2.2 Stimulus/Response Sequences

1. The user submits the start and end locations entered as text in the search bar and clicks "fetch route".
2. The system sends the submitted location text to the OneMap Search API.
3. OneMap Search API returns the corresponding coordinates for the respective start and end locations if valid.
4. The system stores the coordinates and accepts the locations as valid.

4.2.3 Functional Requirements

REQ-1: The system has access to the OneMap Search API.

REQ-2: The user has entered start and end locations as text in the search bar.

REQ-3: Invalid Location:

1. If the OneMap Search API returns no locations based on either the user's start and end location input, no location will be returned/auto completed.

REQ-4: API Error:

1. If the OneMap Search API is unreachable, no suggestion will be displayed.

4.3 Select Mode of Transport

4.3.1 Description and Priority

Allows the user to select between modes of transport.

Priority: High

4.3.2 Stimulus/Response Sequences

1. User enters the main page which displays multiple transport tabs (Best, Driving, Public Transport, Walking, etc).
2. User clicks on the respective tab (mode of transport).
3. The system will query the external API (LTA data mall / onegov) for all modes of transport, regardless of the selected tab.
4. The system displays the results with respect to the selected mode of transport.
5. Users can swap to other transport tabs and view 3 - 4 again.

4.3.3 Functional Requirements

REQ-1: Start and end location validated.

REQ-2: Display the information related to the mode of transport selected by the user.

REQ-3: AF-S3: Data Unavailable (from external API)

1. External API does not return any routes for a specific transport mode (network or invalid).
2. Tab will display the error message "No routes available".

REQ-4: API Error:

1. The system cannot connect to the external API server.
2. The system will display an error message, "No routes available".

4.4 Select Convenience Filter

4.4.1 Description and Priority

The system will display a drop-down of filters which the user can select. These filters modify and directly affect the convenience score in turn affecting routes displayed to the user.

Priority: High

4.4.2 Stimulus/Response Sequences

1. The system will display a drop-down of filters selectable by the user (all filters are selected by default)
2. The user can select/deselect singular, multiple, or no convenience filters by adjusting a slider bar that saves filter preferences.
3. The system will preserve the filter preference across transport tabs.
4. The system stores the user's convenience score filter preference for further calculations

4.4.3 Functional Requirements

REQ-1: Start and end location validated

4.5 Display Convenience Level Score

4.5.1 Description and Priority

The system will calculate and display a convenience level score for each mode of transport. The score will be within the range of 1-10, and they will be displayed to the user by rank based on the convenience level score.

Priority: High

4.5.2 Stimulus/Response Sequences

1. The system will retrieve the user's selected convenience filters.
2. The convenience score is calculated for all routes within the range of 1-10.
3. The convenience score is displayed to the user on each route card.
4. The top three convenience scores from all transport modes are ranked in descending order in a separate "Best" tab.

4.5.3 Functional Requirements

REQ-1: Start and end location validated

REQ-2: External API provides accurate and updated route data

4.6 Display Interactive Map

4.6.1 Description and Priority

User to be provided an interactive map interface which allows users to interact with such as being able to click on locations.

Priority: High

4.6.2 Stimulus/Response Sequences

1. The user opens up the app.
2. Upon opening the application, the main UI will request for OneMapAPI data.
3. API will receive the request and return the main UI with map data.
4. The Main UI will receive the data and display the full map of Singapore with the ability to interact with it.

4.6.3 Functional Requirements

REQ-1: AF-S2: Interactive map is not displayed

1. The user opens the application but the interactive map does not load in, the user will reopen the application.

REQ-2: API Error

1. The Application goes to External API to retrieve info but is met with an error with the API
2. Displays an Error message to inform the user that the API is currently experiencing troubles (API system not functional, API unable to determine information given, etc.)

REQ-3: Users must have an internet connection to be able to connect to the application and the API is working.

REQ-4: Assumes that the Application will only be used in Singapore.

4.7 Retrieve Routes

4.7.1 Description and Priority

Retrieves route options from the OneMap API based on the user's start and end location.

Priority:High

4.7.2 Stimulus/Response Sequences

1. The system sends a request to the OneMap API with the coordinates of the start and end locations
2. The OneMap API responds with route options
3. The system processes the response and prepares the route data for display.

4.7.3 Functional Requirements

REQ-1: API Error:

1. If the OneMap API is unreachable, the system will display an error message: "Unable to retrieve routes. Please try again later."

REQ-2: The system has access to the OneMap API

REQ-3: Start and end locations have been validated

4.8 Save Routes (Itinerary)

4.8.1 Description and Priority

This interface will show the user their saved/favourited routes for ease of use if this is a common route they take.

Priority: Low

4.8.2 Stimulus/Response Sequences

1. After the user has displayed the generated itinerary, the user will click the 'Save' button located in the side bar.
2. Itinerary and filter will be saved to the Saved Routes interface for future reference or use
3. User will be able to access their saved route through the saved routes tab

4.8.3 Functional Requirements

REQ-1: AF-S3 No Route option saved

1. If the user did not save any routes previously, system will not display any saved routes.
2. User to select a Starting Point and Destination
3. Users can choose their desired route and click the bookmark (save) icon.

REQ-2: User is logged into their account and has chosen a Starting Point and Destination with the routes and itinerary generated

4.9 Interactive Map Start/End Point Selector

4.9.1 Description and Priority

User can pick their Starting Point or Destination by clicking directly on the interactive map

Priority: Low

4.9.2 Stimulus/Response Sequences

1. User will click on a location on the interactive map
2. Information related to the location will pop up as a bubble (longitude and latitude)
3. Application will automatically fill in the respective position

4.9.3 Functional Requirements

REQ-1: AF-S1 Location is an invalid point in the map

1. User clicks on a location which has no valid coordinates
2. Display Error Message

REQ-2: Interactive map API is not unreachable

REQ-3: The map is displayed and clearly visible to the user

4.10 Display Driving Route

4.10.1 Description and Priority

Display route results in driving mode.

Priority: Medium

4.10.2 Stimulus/Response Sequences

1. The user selects driving as the preferred mode of transport.
2. The system retrieves the list of available driving routes from the parent use case Retrieve Routes.
3. The system displays route cards for each driving route (e.g., including distance, travel time).
4. The system plots the driving routes on the interactive map.
5. The user may select a route card to view detailed instructions

4.10.3 Functional Requirements

REQ-1: AF-S3

1. If there are no possible driving routes, the system displays “no routes available” in the driving tab.

REQ-2: User inputs start and end destination and selects driving as the preferred mode of transport.

4.11 Display Nearby Carparks Info

4.11.1 Description and Priority

Display nearby carparks info in driving mode.

Priority: Medium

4.11.2 Stimulus/Response Sequences

1. The user selects driving as mode of transport.
2. The system gets driving routes by querying the external API.
3. The system retrieves possible routes from external API.
4. For each route, the systems retrieve information about nearby carparks such as availability
5. The system displays nearby carparks as pins in the map .
6. If users click on the pins, a window pops up showing the detailed information about the carpark.

4.11.3 Functional Requirements

REQ-1: AF-S6

1. If there is a lack of data about some data, like parking rates, the system displays that the information is not available.

REQ-2: EX1 If the API is down

1. The system informs the user that the service is temporarily unavailable and includes a Retry action.

REQ-3: User inputs start and end destination and selects driving as the preferred mode of transport.

4.12 Display Traffic Incident Info

4.12.1 Description and Priority

Retrieve traffic incidents information on the route.

Priority: Medium

4.12.2 Stimulus/Response Sequences

1. The system gets possible routes by querying the external API.
2. For each route, the system queries external API and filters incidents that directly impact the route. For driving mode, the system queries traffic accidents on the road.
3. The system displays that information on the map with pins on the affected route segments and names of roads affected on the route result card.

4.12.3 Functional Requirements

REQ-1: AF-S3:

1. No incident found, the system does not display any pins on the route.

REQ-2: EX1: If the API is down,

1. The system informs the user that the service is temporarily unavailable and includes a Retry action.

REQ-3: User input start and end destination.

4.13 Display Public Transit Route

4.13.1 Description and Priority

Show buses/trains needed with transfer points.

Priority: High

4.13.2 Stimulus/Response Sequences

1. The user selects public transit as the preferred mode of transport.
2. The system retrieves the list of available public transit routes from the parent use case Retrieve Routes.
3. The system displays route cards for each public transit route (e.g., including distance, travel time).
4. The system plots the public transit routes on the interactive map.
5. The user may select a route card to view detailed instructions such as buses and MRT to take as well as walking segments

4.13.3 Functional Requirements

REQ-1: AF-S2

1. Services are unavailable due to the time of day, then the system displays that error to the user.

REQ-2: AF-S1

1. User inputs a location outside of Singapore, the system then displays the message to the user.

REQ-3: User input start, end destination and select public transport mode.

4.14 Display Public Transit Info

4.14.1 Description and Priority

Display detailed travel instructions and number of transfers and total fare cost

Priority: High

4.14.2 Stimulus/Response Sequences

1. User selects a particular itinerary in the public transit mode.
2. For each route in the itinerary:
 - a. System retrieves scheduled departure/arrival.
 - b. System retrieves real-time updates such as platform density, average bus waiting time (where applicable) and estimated travel time.
3. API returned itineraries containing buses, MRT and walking segments.

4.14.3 Functional Requirements

REQ-1: AF-S2

1. Platform information is missing, the system shows that the information is not available.

REQ-2: EX1 If the API is down

1. The system informs the user that the service is temporarily unavailable and includes a Retry action.

REQ-3: User inputs start and end destination and selects the public transport mode.

4.15 Display Walking Route

4.15.1 Description and Priority

Shows Walking route.

Priority: High

4.15.2 Stimulus/Response Sequences

1. The user selects public walking as the preferred mode of transport.
2. The system retrieves the list of available walking routes from the parent use case Retrieve Routes, and the forecast.
3. The system displays route cards for each walking route (e.g., including distance, travel time, weather forecast...).
4. The system plots the walking routes on the interactive map.
5. The user may select a route card to view detailed instructions on the walking directions.

4.15.3 Functional Requirements

REQ-1: User inputs start and end destination and selects walking as the preferred mode of transport.

REQ-2: Start and End locations are validated.

4.16 User Login

4.16.1 Description and Priority

The back-end and front-end systems are involved in managing authentication services within the application. It is used in validating entered credentials and providing appropriate error messages otherwise.

Priority: High

4.16.2 Stimulus/Response Sequences

1. The user opens the website.
2. The system pops up a login form with email and password and a login
3. The system validates the input credentials (e.g valid email, password length)
4. The system queries and validates with the account database to check account validity
5. If the account is valid, the system provides a login session token and redirects the user to the main page.

4.16.3 Functional Requirements

REQ-1: Invalid Login Credentials - If the user provides invalid login credentials, an error message with the text "invalid login credentials, please try again" message below the associated fields.

REQ-2: Attempt Lockout - After 5 failed login attempts, the account is temporarily locked and an email is sent to the user's email on suspicious login activity.

REQ-3: User must already have an existing account within the application database.

REQ-4: The system is already connected to the account database.

REQ-5: The user has an existing valid email.

REQ-6: The application in production must support HTTPS and have valid authority certificates.

REQ-7: The password in transit and in storage must be salted and hashed for data security.

4.17 Register User

4.17.1 Description and Priority

Registration of a user with no existing account, providing credentials such as email address, password and username for added features of the app.

Priority: High

4.17.2 Stimulus/Response Sequences

1. User clicks on the sign-up button on the login screen.
2. System will pop up a registration form with the following details:
 - a. Name
 - b. Email Address
 - c. Password
3. System validates entered data with the following criteria.
4. Password
 - a. 8 Characters
 - b. At least 1 numerical digit
 - c. At least 1 special symbol / character
5. Email Address
 - a. Presence of @ symbol
6. System sanitises and prevents encoding of HTML tags.
7. Upon successful validation, create a new user and store it within the system's account database.
8. The system then redirects the user to the login page.

4.17.3 Functional Requirements

REQ-1: AF-S3 Weak / Invalid Password/email

1. User inputs an invalid password/email.
2. System displays an error message below the respective field.

REQ-2: AF-S3 Account Exists

1. User inputs an email that already exists in the database.
2. Upon pressing login, the system displays an error.

REQ-3: Invalid Characters - Reject HTML-enclosed tags such as <script> and provide an error message for illegal use of characters.

REQ-4: The application in production must support HTTPS and have valid authority certificates.

REQ-5: The password in transit and in storage must be salted and hashed for data security.

REQ-6: The system must sanitise and encode all user inputs to prevent XSS.

REQ-7: Error messages must not reveal anything about the system.

REQ-8: The user has an existing valid email.

REQ-9: User is not logged in.

4.18 Change Password

4.18.1 Description and Priority

Allow users to change their password under their profile page.

Priority: High

4.18.2 Stimulus/Response Sequences

1. User presses the 'Change Password' button under their profile page.
2. User keys in their current password as well as their new password.
3. New password must meet password requirements and must match, otherwise an error message is returned.
4. The system updates the user's account in the database with the updated password.

4.18.3 Functional Requirements

REQ-1: Authentication Services Down - Provide an error code and display the message "Authentication Services are unreachable, please try again later" message.

REQ-2: Invalid Characters / Not meeting minimum requirements - Reject HTML-encoded tags such as <script> and provide an error message for illegal use of characters.

REQ-3: The domain of the sending email (email service that sends the reset link) must be of a unique and trusted domain.

REQ-4: The user has an existing valid email in the database.

REQ-5: The system is already connected to the account database.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

1. The Web Application must load and display the route within 9 seconds of user input under normal network conditions.
2. The Web Application shall handle up to 100 concurrent users without significant performance degradation.
3. API calls must have a response time of under 2 seconds for 95% of requests under normal conditions.

5.2 Safety Requirements

1. When displaying real-time safety information (e.g. Traffic Incidents, Road Closure or MRT Service Disruption), the system must explicitly indicate the time of such obstruction and the direction it affects received from the source API.

5.3 Security Requirements

1. The system must use HTTPS to encrypt all data transmitted between the client and server.
2. The system shall sanitise all user inputs to prevent injection attacks (e.g., XSS, SQL injection).
3. The system shall comply with Singapore's Personal Data Protection Act (PDPA) if user data is stored or processed.
4. The system should terminate user sessions when idle for more than 30 minutes in the application.
5. Codebase must not directly contain private API keys, passwords, or other confidential information.

5.4 Software Quality Attributes

1. Code will always be accompanied by the relevant code comments indicating at least the purpose of the following block of code, to allow for interpretability, code reusability and future maintenance.
2. The system will display the appropriate error or success messages when actions are done by the user to improve the testability of the product.
3. To ensure reliability, the application should only crash or fail to load at most 1% of the time.

4. The application must be compatible with modern web browsers (e.g. Chrome, Firefox, Safari, Edge) on both desktop and mobile without major differences.

5.5 Business Rules

Business rules provide a framework for the necessary functionality that “EasyGo” will require to ensure that they help to ensure that the system operates within the intended legal and ethical boundaries.

5.5.1 Data Privacy

1. All user data must be governed by the legal standards of the Singapore Personal Data Protection Act (PDPA).
2. For the safety and protection of the user, active sessions must not exceed 30 minutes of continuous inactivity.

5.5.2 Operational Boundary

1. EasyGo location information is only available within Singapore’s geographical boundaries.
2. All distances and measurements presented to the user must conform to the International System of Units (SI).

5.5.3 Transactional Integrity

1. All user input must be safe and not compromise the system's integrity.
2. The application must be deployed where data transmission undergoes mandatory encryption (HTTPS).

5.5.4 Function Restrictions

Table 5.5.4.1 showcases the functions that a user of EasyGo is able to perform.

Login/Signup	inputLogin() resetPassword() inputRegister()
Search/Result	setStartLocation() setEndLocation() saveItinerary() displayInteractiveMap() selectLocation()
Side Route Panel	selectItinerary()
Saved Routes	clickSave() saveRoute()

Table 5.5.4.1

6. Other Requirements

6.1 Internationalisation Requirements

1. Initially, EasyGo will only support Singapore-based locations. Future updates will consider expanding the location data to other countries for a broader audience to use.
2. Distance representation for routes should be in the International System of Units (SI).

6.2 Legal Requirements

1. EasyGo must comply with the Personal Data Protection Act (PDPA) when handling user's personal data.
2. Data collected is only used for the purpose for which consent was given.

6.3 Reuse Objectives

1. EasyGo is designed with component reuse and modularity to reduce future development time and maintenance effort.
2. API service modules must be designed to be stateless and reusable by all features.

6.4 Accessibility Requirements

1. EasyGo will adhere to basic accessibility standards to ensure the application is usable by the broadest range of individuals.
2. EasyGo must have Web Content Accessibility Guidelines (WCAG) 2.1 Level AA for people with disabilities.

Appendix A: Glossary

Term	Definition
User	The primary actor, typically a commuter in Singapore, who uses the SG Transport Planner Web App to plan routes in Singapore.
External APIs	Refers to APIs provided by external sources such as LTA Datamall, data.gov.sg and OneMap. These APIs provide real-time data for routes, traffic, carparks, and transit schedules required for the web application to function.
System	The backend of the web application
System Log	A recorded history of events within a computer or transport system.
Data Error (API Error)	An error code generated regarding external API access
Error Code	An identifier that indicates a specific problem in a system
Convenience Level Score	Refer to the Appendix A
ETA	Time taken to arrive at destination in minutes
User Profile	A personal profile to store information, preferences, and history including saved routes.
Interactive Map	An interactive map where users can pan and view points of interest.
Route	The directions from source to destination location based on the transport mode selected
Alternate Route	An alternate path to reach the same destination
Start, End Destination	Location of starting point and ending points taken from user's input
Travel Time	Time taken to travel from start to end destination.
Transport Mode	The type of transport to take. Available options include: 1. Driving 2. Public Transit 3. Walking
Peak Hour	The time when traffic is most crowded, more specifically 7AM - 9AM and 6PM - 8PM

Driving Mode	A transport mode where the person uses a private motor vehicle (e.g., car, motorcycle).
Carpark	A designated area where users can park their vehicles.
Traffic Incident Information	Information on traffic incidents that happen along the route which impact the route.
Traffic Disruption	An event where traffic is disrupted either by traffic incidents or by traffic congestion.
Public Transit Mode	Category of transportation service that is publicly available to multiple passengers, e.g bus and MRT.
Bus Wait Time	Waiting time at bus station
MRT Service Disruption	An event where Train Service along a certain line is disrupted, affecting ETA and route suggestion capability.
Station Crowded Level	A measure of how many people are present at a station within a given time (Low / Med / High)
Valid Location	Existing accessible local area in Singapore provided by external API
Carpark Availability	Presence of vacancies in each carpark lot.

Convenience Score Considerations

Related to All / Walking

- Time taken to reach the destination (More time, less convenient)
- Amount of Walking (More walking, less convenient)

Public Transport Related

- Number of transfers (More transfers, less convenient)
- Crowd Level at train station. (Denser population, less convenient)
- Bus wait time (Longer wait times, less convenient)
- Fare Cost

Driving Related

- Carpark Availability near destination (Less availability, less convenient)

Normalised Time Equations

Each normalised value will be scaled by 10 (multiply by 10)

- Time taken to reach the destination (More time, less convenient)

$$\text{Normalized Time} = \frac{\text{Time Taken} - \text{Min Time}}{\text{Max Time} - \text{Min Time}}$$

- Amount of Walking (More walking, less convenient)

$$\text{Normalized Walking} = \frac{\text{Distance} - \text{Min Distance}}{\text{Max Distance} - \text{Min Distance}}$$

- Number of transfers (More transfers, less convenient)

$$\text{Normalized Transfers} = \frac{\text{Transfers}}{\text{Max Transfers}}$$

- Crowd Level at train station. (Denser population, less convenient)

$$\text{Normalized Crowd} = \frac{\text{Crowd Percentage}}{100}$$

- Bus wait time (Longer wait times, less convenient)

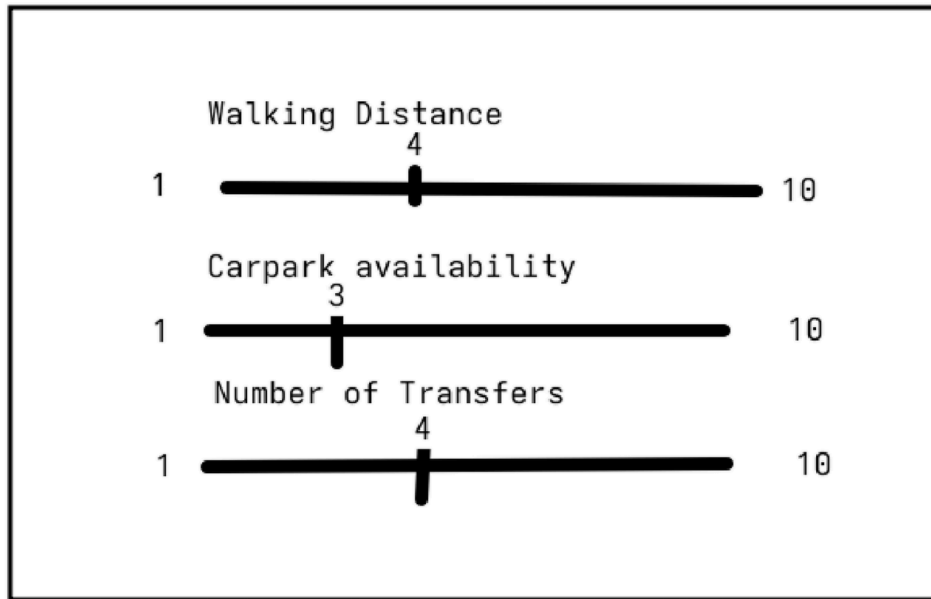
$$\text{Normalized Wait Time} = \frac{\text{Wait Time}}{\text{Max Wait Time}}$$

- Carpark Availability near destination (Less availability, less convenient)

$$\text{Normalized Carpark} = 1 - \frac{\text{Availability Percentage}}{100}$$

Calculate Weightage

The importance of each factor is determined between a value of 1 to 10 and is selectable by the user. For example, if the user selects the value 4 for walking distance, the value 3 for carpark availability and the value 4 for number of transfers, the weight of each factor is calculated using the formula shown below.



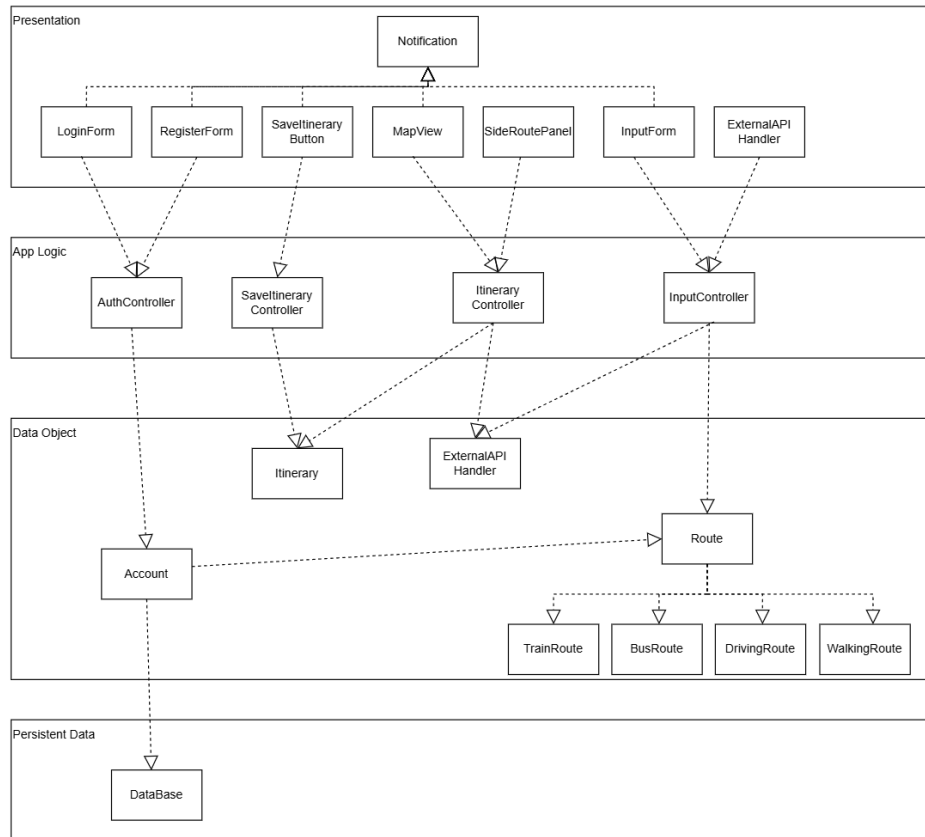
$$\text{Weight}(W_i) = \frac{\text{Selector Value}(S_i)}{\text{Sum of All Selector Values}(\sum S_j)}$$

Each normalised score will be multiplied by its respective weightage.

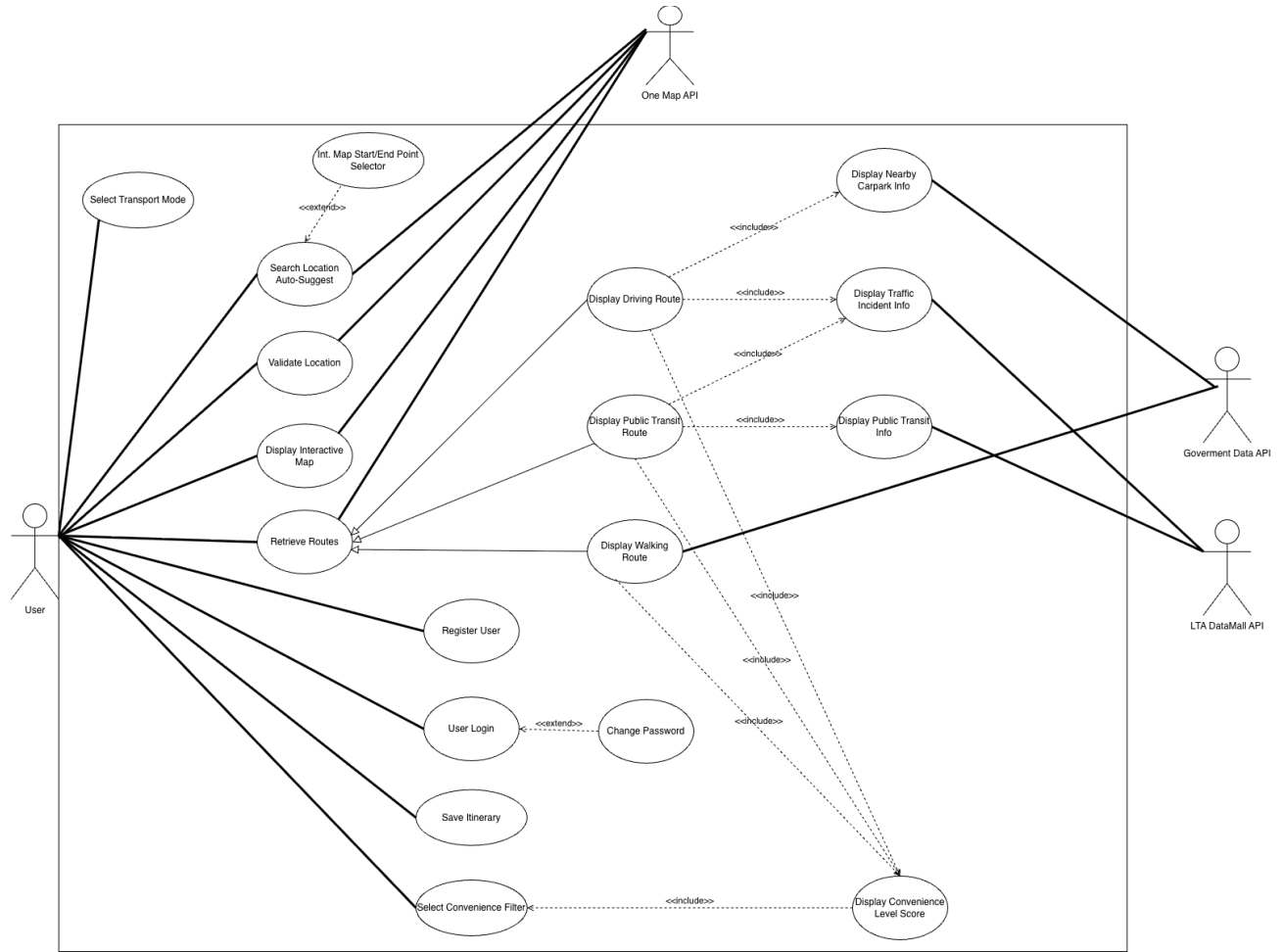
Convenience Score

All normalised scores will be added together to find the inconvenience score.
The convenience score will be (10 - inconvenience score).

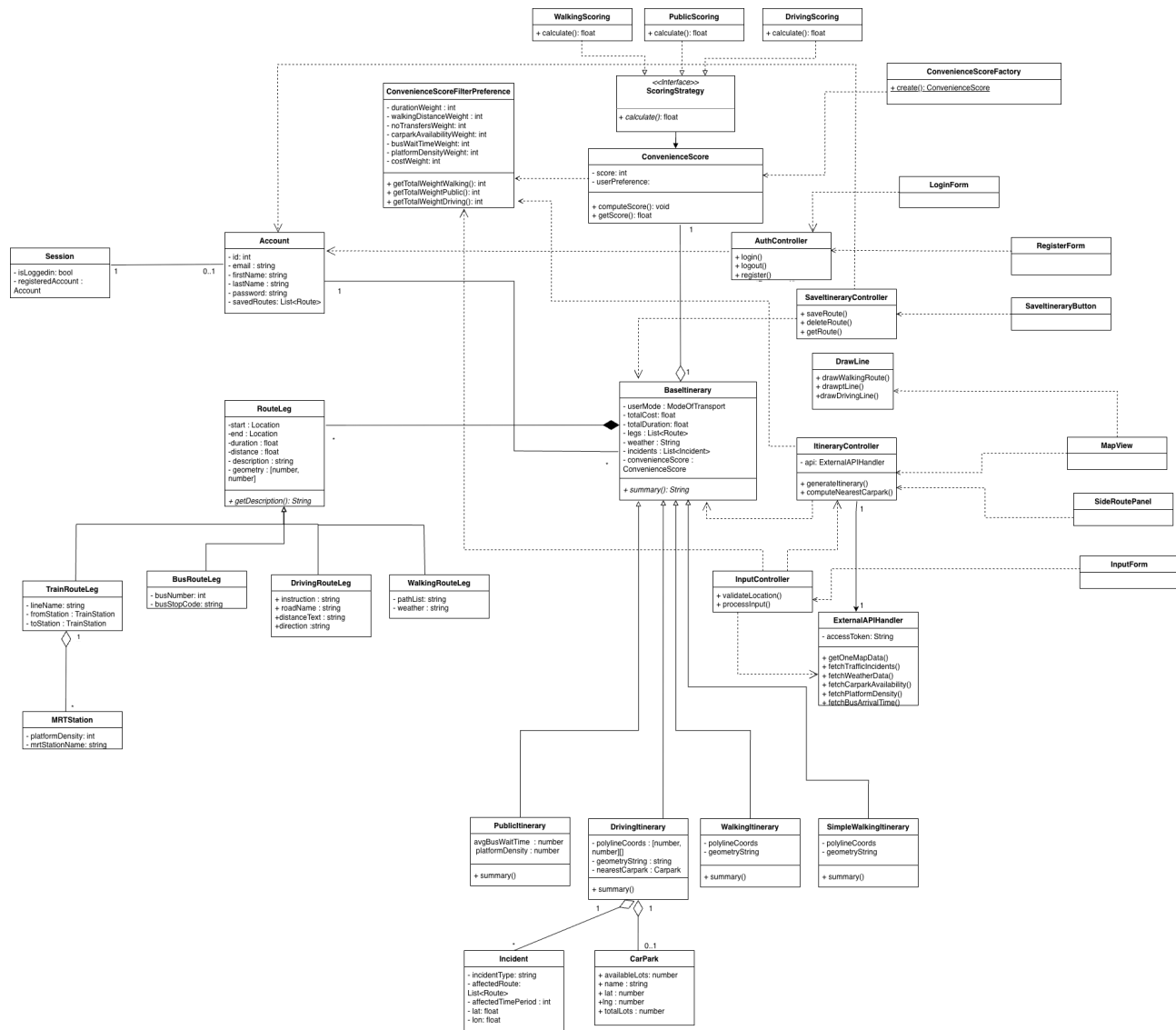
Appendix B: Analysis Models



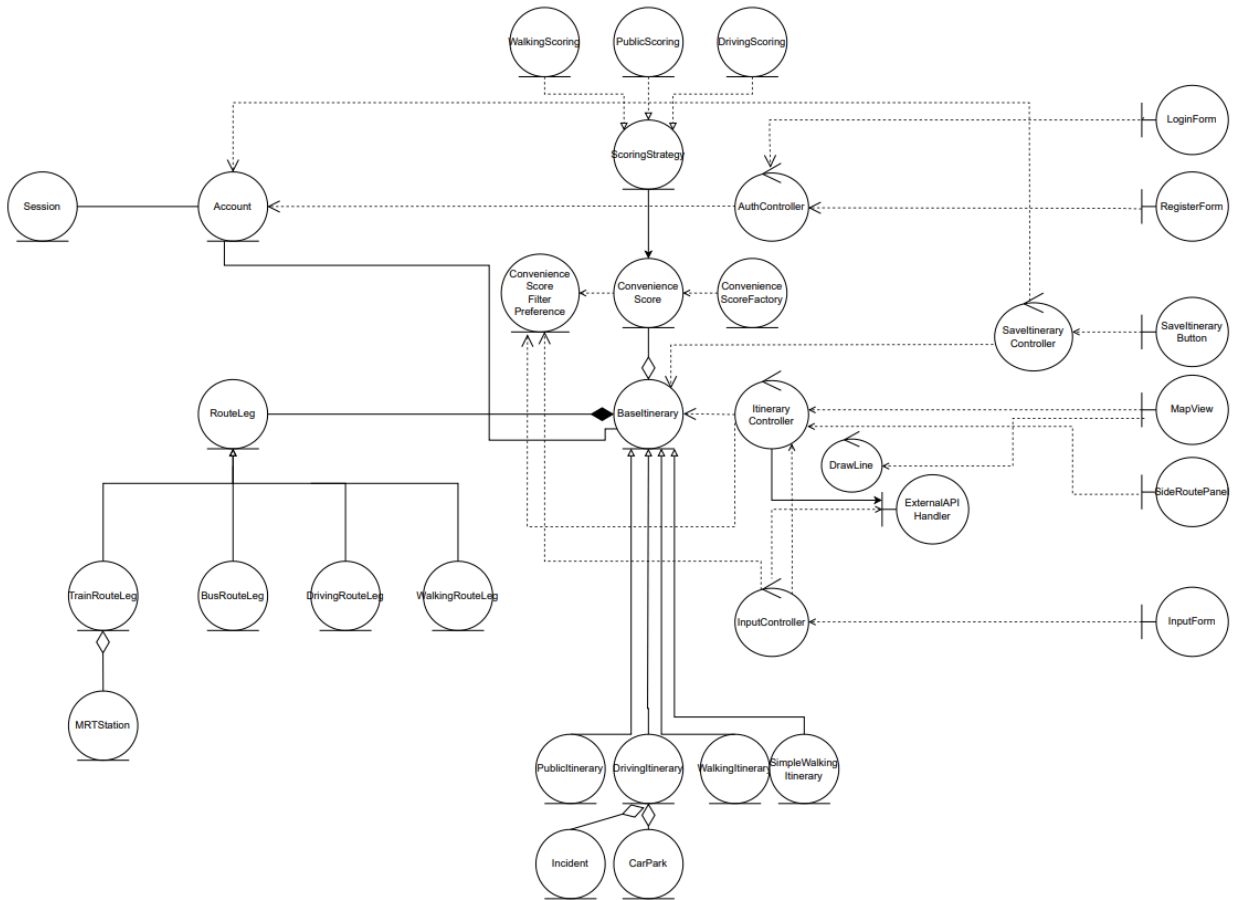
Architecture Diagram



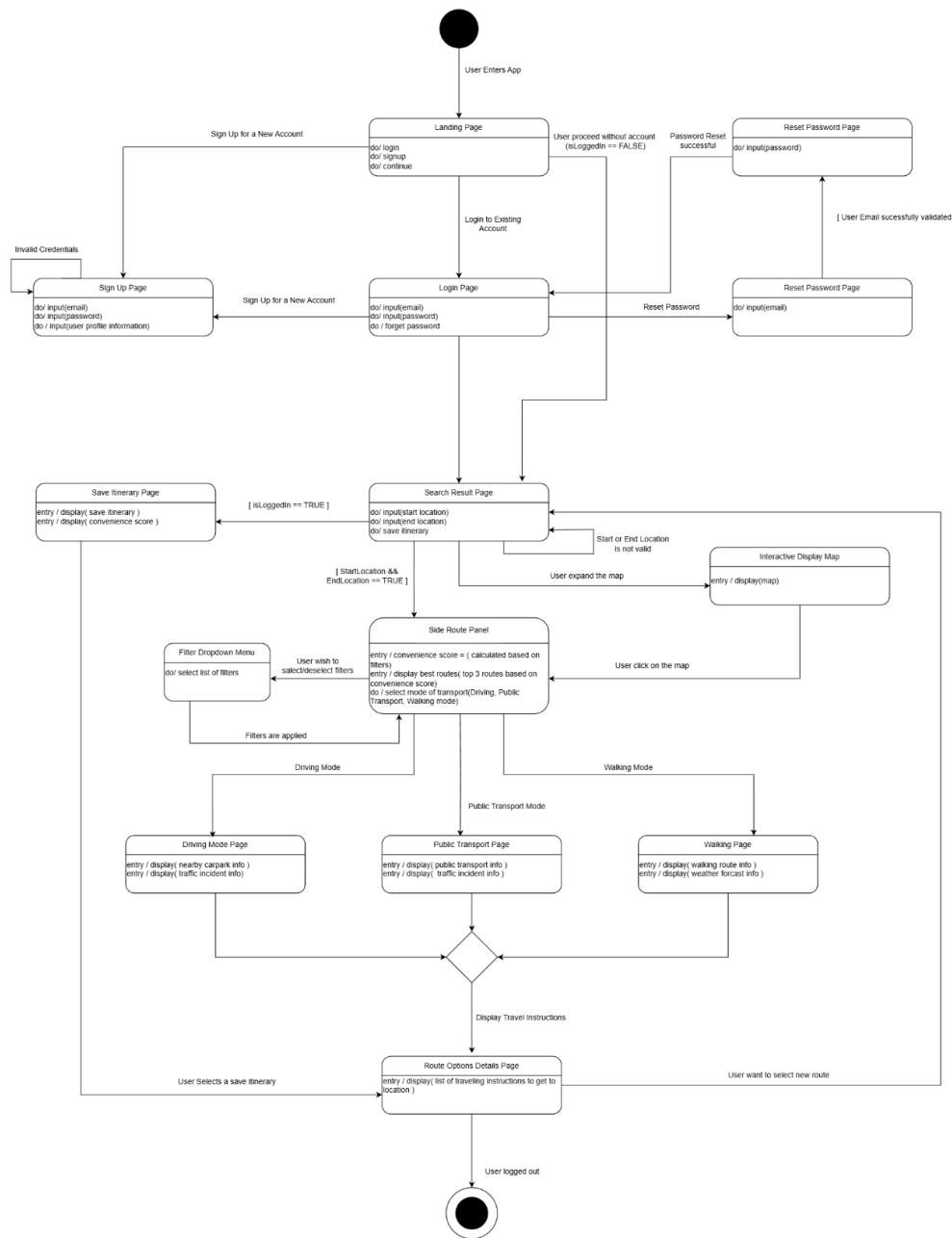
Use Case Diagram



Class Diagram



Stereotype Class Diagram



Dialog Map

Appendix C: To Be Determined List

There are no TBD items in this version of Software Requirements Specifications.

Source: http://www.frontiernet.net/~kwiegers/process_assets/srs_template.doc